Granular ball local consensus for graph-free node classification under label noise

Silei Cao[a], Kun Wang[a], Jingyu Zhu[b], Tianxing Wang[c] and Bing Huang[a,*]

[a]School of Computer Science, Nanjing Audit University, Nanjing, 211815, PR China
[b]School of Statistics and Data Science, Nanjing Audit University, Nanjing, 211815, PR China
[c]School of Business, Nanjing Audit University, Nanjing, 211815, PR China

ARTICLE INFO

Keywords:
label noise
node classification
graph-free inference
graph contrastive learning
granular ball computing
soft labels

ABSTRACT

Graph-free node classification enables prediction from node attributes when graph relations are unavailable or costly to access at deployment. However, existing noise-robust graph methods generally retain graph-dependent inference, whereas graph-free methods have largely been studied with clean training labels. To address this gap, we propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. GB-LCSS learns node representations through label-free graph contrastive learning, freezes them, and recursively groups nodes into granular balls. For each training node, the observed labels of the other training nodes in its ball form a leave-one-out consensus. A weight determined by their agreement blends this consensus with the node's observed label to produce a soft target for a multilayer perceptron (MLP). This design decouples region construction from noisy-label aggregation and uses local evidence to reduce the influence of isolated annotation errors. Across five datasets and five noise settings, GB-LCSS achieves the highest five-seed mean accuracy in 24 of the 25 settings, compared with seven noisy-label baselines that use the same deployed MLP architecture. Its soft targets also have lower mean Brier scores than the observed one-hot labels in all 25 settings.

1. Introduction

Node classification is a fundamental task in graph learning, in which node attributes and relationships jointly support prediction. Graph neural networks (GNNs) combine these two sources of information by aggregating representations from neighboring nodes [1]. This dependence on graph structure, however, complicates deployment: each prediction requires access to the relevant neighborhood, and retrieving and aggregating multi-hop neighbor information adds computational cost. In applications with missing relations, privacy restrictions, or low-latency requirements, such access may be unavailable or impractical. These constraints have motivated graph-free node classification, in which graph structure is used during training but the deployed classifier receives only node attributes. Graph-MLP [2] and GLNN [3] demonstrated this approach, and later work such as SimMLP [4] and FF-G2M [5] refined what structural knowledge is transferred to the multilayer perceptron (MLP) and how. These methods have been developed and evaluated with clean training labels.

In practice, however, training labels obtained through manual annotation, automated rules, or multiple sources may contain errors, and deep models can fit such corrupted annotations [6]. On graphs, message passing may propagate the influence of mislabeled nodes to their neighbors [7]. Noise-robust graph methods use topology to augment supervision [8], regulate training with noisy labels [9], or select reliable samples [10], but they retain graph access at inference. Graph-free classifiers face a related challenge: errors in the labels used to train an MLP or its teacher can carry over to the deployed model. We study how the training graph can improve noisy-label supervision for a classifier that uses only node attributes at inference.

*Corresponding author.
Email addresses: mp2509053@stu.nau.edu.cn (S. Cao); kun_wang@nau.edu.cn (K. Wang); jingyuzhu77@gmail.com (J. Zhu); wangtianxing@nau.edu.cn (T. Wang); huangbing@nau.edu.cn (B. Huang)

One way to address this challenge is to combine an observed label with evidence from nearby training nodes. When their labels provide consistent evidence for the underlying class, aggregating them can reduce the influence of isolated annotation errors. Related methods use neighborhood consistency to identify potentially mislabeled samples or encourage agreement between neighboring predictions [11, 12]. For graph data, a graph contrastive representation learned from node attributes and connectivity without class labels provides a basis for identifying such local groups [13]. The remaining challenge is to form local regions from the representation geometry before aggregating labels, so that annotation errors do not determine which nodes contribute evidence.

Granular ball computing offers one way to construct such regions. It groups samples into coarse-grained units summarized by their centers and radii [14]. Subsequent studies have investigated granular ball sampling for noisy or imbalanced classification [15] and refined adaptive ball generation [16]. Unlike supervised granular ball classifiers that use label purity to guide splitting, GB-LCSS forms balls from representation geometry under size and ball-count constraints before aggregating training labels.

Building on this idea, we propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. As shown in Fig. 1, GB-LCSS first learns node representations through label-free graph contrastive learning and freezes them. It then recursively applies 2-means to the frozen node representations, assigning each node to exactly one granular ball. For each training node, the observed labels of the other training nodes in its ball provide a leave-one-out consensus. A weight determined by their agreement blends this consensus with the node's observed label to form a soft target. An MLP trained on these targets receives only node attributes at inference.

The main contributions of this paper are summarized as follows:

(1) We introduce GB-LCSS for noisy-label node classification with graph-free inference. It organizes nodes into granular balls in a frozen, label-free graph representation, thereby separating region construction from noisy-label aggregation.

(2) We construct node-specific soft targets by combining each observed label with the consensus of the other training nodes in its ball. Under independent symmetric label noise, we analyze how the number of votes and the underlying class distribution affect the reliability of local voting. We also derive an exact decomposition of the squared error of the soft target and a sufficient condition under which it has lower expected error than the observed label.

(3) We evaluate GB-LCSS on five datasets under five noise settings against noisy-label learning methods and graph-assisted MLP methods. Controlled comparisons examine grouping, representation learning, target construction, and parameter sensitivity. A graph-teacher extension evaluates the same targets against graph-to-MLP distillation methods.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents GB-LCSS, analyzes local-consensus supervision, and introduces the graph-teacher extension. Section 4 describes the experimental setup, comparative results, ablation studies, extensions, and discussion. Section 5 concludes the paper.

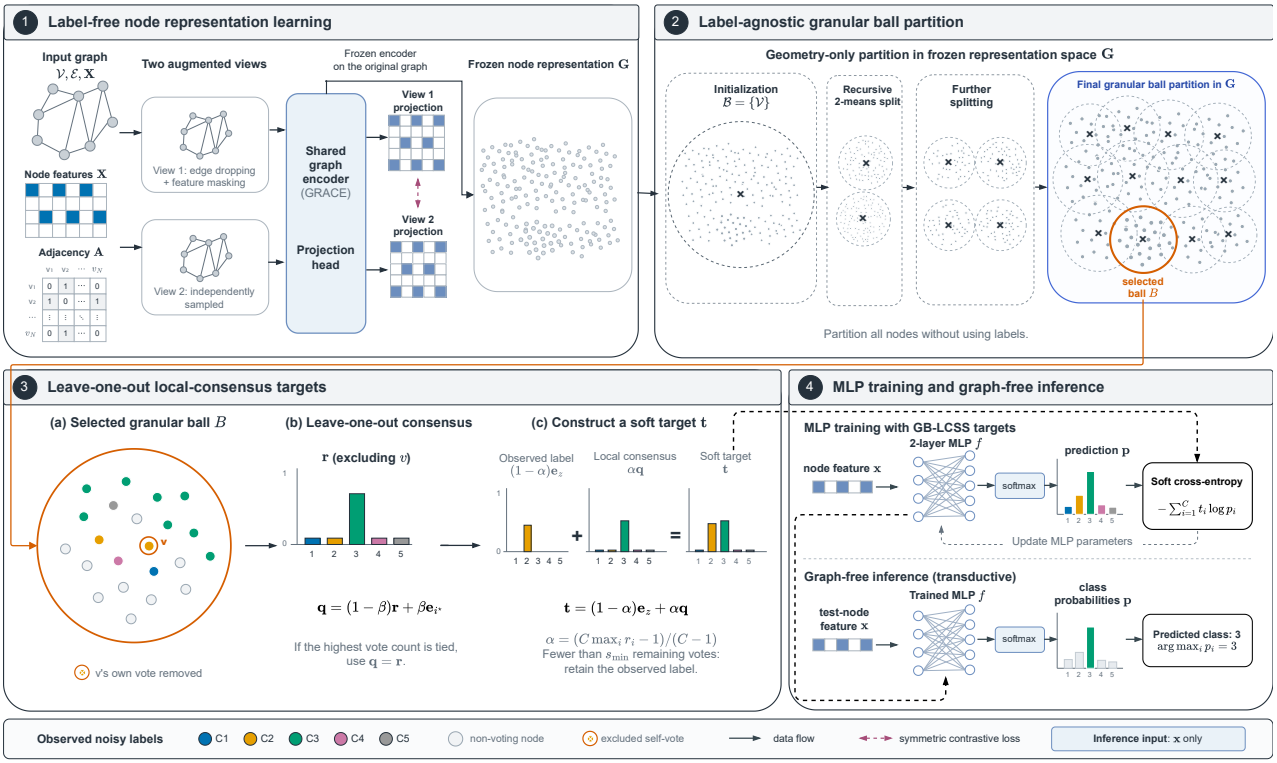


Figure 1: Overview of GB-LCSS. Stage 1 learns a label-free representation, and Stage 2 constructs granular balls in that representation space. Stage 3 shows an enlarged view of a representative ball to illustrate how a local-consensus target is formed. Stage 4 trains the deployed MLP with the resulting soft targets. Hollow nodes do not vote. Ball outlines are schematic, and each node belongs to exactly one ball. For clarity, node and ball indices are omitted in the schematic.

2. Related work

2.1. Learning with noisy labels

Methods for learning with noisy labels reduce the influence of corrupted annotations by selecting training samples, encouraging consistent predictions, or modifying the training objective. Sample-selection methods use the training loss to assess label reliability: Co-teaching [17] exchanged small-loss samples between two networks, and JoCoR [18] additionally encouraged prediction agreement. Local relationships offer another source of evidence. Deep k-NN [12] filtered out samples whose labels disagreed with those of their neighbors, while neighbor-consistency regularization [11] encouraged the prediction for each sample to match those of its neighbors.

Other methods address label noise by modifying the training loss. SCE [19] combined cross-entropy with reverse cross-entropy, ELR [6] regularized training to limit noise memorization, and LogitClip [20] constrained logit norms. These strategies handle annotation noise without explicitly modeling graph relations between samples.

2.2. Noise-robust learning on graphs

Graph structure can provide additional evidence for assessing label reliability, while message passing may also spread the influence of supervision errors [7]. UnionNET [21] used neighborhood-based label estimates for reweighting and correction, NRGNN [8] augmented supervision through structural information and pseudo-labeling, and RTGNN [9] coordinated two networks to regulate noisy supervision.

Label reliability can also be assessed through graph topology and model behavior. TSS [10] used topological sample selection, ProCon [22] used peer-informed label consistency, and GNN-CFGD [23] combined coarse- and fine-grained sample division to address label noise and sparsity.

Beyond assessing label reliability, several methods have constructed additional supervision. DND-NET [24] used neighborhood uncertainty to reweight pseudo-label supervision, and RNCGLN [25] combined graph contrastive learning with pseudo-graphs and pseudo-labels to address graph and label noise. Prototype-Guided Supervision [26] formed class-level prototypes by aggregating the representations of nodes with the same observed label, while SAMTD [27] improved supervision through multiple graph teachers.

Other methods modify neighborhood aggregation or the training objective. MILAB [28] combined information-bottleneck-guided aggregation with adversarial consistency, and HALF [29] fused low- and high-pass losses according to local homophily to identify reliable labels. In addition, NoisyGL [7] established a unified evaluation framework for learning with noisy labels on graphs.

2.3. Graph-free node classification

Methods for graph-free node classification exploit graph information during training while avoiding online message passing at inference. Graph-MLP [2] incorporated adjacency information through a neighborhood-contrastive objective, whereas GLNN [3] distilled predictions from a GNN teacher into an MLP. Both methods deploy attribute-only MLPs. GSDN [30] used neighborhood self-distillation, SimMLP [4] aligned GNN and MLP representations without supervision, and SA-MLP [31] investigated structure-aware distillation. These methods differ in how graph information guides training and in whether precomputed structural features are retained at inference.

Subsequent distillation methods differ in the teacher information they transfer. FF-G2M [5] separated low- and high-frequency knowledge, KRD [32] assessed knowledge reliability through the stability of prediction entropy under noise perturbations, and VQGraph [33] encoded local structures as discrete tokens. PGKD [34] used class prototypes to guide structural knowledge distillation from GNNs to MLPs, whereas TINED [35] combined teacher parameter injection with Dirichlet-energy distillation. SALE-MLP [36] investigated structure-aware latent representations. Yu et al. [37] theoretically analyzed how label alignment in distillation allows an MLP to learn the implicit graph bias of a GNN teacher.

A related line of work uses graph structure to construct soft training targets. PosteL [38] constructed posterior soft labels from neighborhood labels and graph-wide label statistics. ALS [39] used neighborhood label distributions and adaptively updated smoothed targets during graph training, while SALS [40] incorporated structural information into the target label distribution. These methods use adjacency-defined neighborhoods to construct or refine training targets. GB-LCSS also modifies the training targets, but it defines regions by proximity in a frozen, label-free representation and excludes each node's own label from the consensus that adjusts it.

2.4. Graph contrastive representation learning

Self-supervised graph learning derives training signals from node attributes and graph connectivity [41]. Graph contrastive learning generates different views of a graph by perturbing its structure or attributes. A common node-level

objective brings the representations of the same node across views closer together while distinguishing them from those of other nodes. The resulting node representations provide a basis for comparing nodes without using their observed class labels.

GRACE [13] implements this approach through random edge dropping and feature masking, with a shared graph encoder processing the two views. Because its objective uses no class labels, GB-LCSS can compute and freeze the GRACE representation before any training label is used; the observed training labels enter only afterwards, when soft targets are constructed within the resulting groups.

2.5. Granular ball computing

Granular ball computing represents samples using coarse-grained hyperspherical regions, which GBkNN used to classify query samples [14]. Granular ball sampling extended this representation to sample selection for noisy or imbalanced classification [15], and subsequent work refined adaptive ball generation [16]. MGNR [42] used granular ball representations to construct multi-granularity neighborhood relationships for classification and clustering.

On graphs, supervised granular ball coarsening [43] and GBGC [44] used granular regions to construct coarsened graphs, reducing the size of the graph on which a GNN operates. The methods above use granular regions for classification, clustering, sample selection, or graph coarsening. In supervised granular ball classifiers and sampling methods, label purity can guide ball generation [14–16]. In contrast, GB-LCSS uses granular balls to organize supervision: a geometric criterion forms them in a frozen, label-free graph representation, and the observed training labels are then aggregated within each ball. Granular ball membership determines which other training nodes contribute to the soft target of each node. These targets transfer graph information into the supervision of an MLP that receives only node attributes at inference.

3. Method

3.1. Problem setting and notation

This subsection defines the learning task and introduces the notation used to describe GB-LCSS.

Consider an attributed graph with node set $\mathcal{V} = \{v_1, \dots, v_N\}$, edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$, adjacency matrix $\mathbf{A} \in \{0,1\}^{N\times N}$, and attribute matrix $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N]^\top \in \mathbb{R}^{N\times d}$, where $\mathbf{x}_n$ is the attribute vector of node v_n.

Each node belongs to one of $C \geq 2$ classes, indexed by $i \in \{1, \dots, C\}$. The set $\mathcal{V}_L \subseteq \mathcal{V}$ contains the labeled training nodes. Node v_n has a clean label y_n; if $v_n \in \mathcal{V}_L$, its observed label z_n may be corrupted. Both labels take values in $\{1, \dots, C\}$. The method uses observed labels, whereas clean labels serve as the reference for analysis and evaluation.

To distinguish quantities associated with different objects, we use n and ℓ as node indices, j as a granular ball index, and i as a class index. Bold lowercase and uppercase letters denote vectors and matrices, respectively, and calligraphic letters denote sets or collections. A second subscript selects a class component; for example, $p_{n,i}$ is the class-i entry of the prediction vector $\mathbf{p}_n$ defined below. Table 1 collects the main symbols used in the method.

During training, the complete $\mathbf{X}$ and $\mathbf{A}$, including the attributes and topology of test nodes, are available for label-free representation learning and target construction, while test labels remain hidden. The goal is to learn an MLP f whose

Table 1
Main notation.

Symbol	Meaning
Graph and data	
$\mathcal{V}, \mathcal{E}$	Node set and edge set
N, C	Number of nodes and number of classes
n, ℓ	Node indices in $\{1, \dots, N\}$
j	Granular ball index
i	Class index in $\{1, \dots, C\}$
$\mathbf{A}, \mathbf{X}$	Adjacency matrix and node attribute matrix
$\mathbf{x}_n$	Attribute vector of node v_n
$\mathcal{V}_L$	Set of labeled training nodes
y_n, z_n	Clean and observed labels of node v_n
η	Label corruption probability
Representations and granular balls	
$\mathbf{G}, \mathbf{g}_n$	Frozen normalized GRACE representation matrix and vector for node v_n
d, h	Attribute dimension and GRACE representation dimension
$\mathcal{B}, B_j$	Collection of granular balls and its j-th member
$\boldsymbol{\mu}_j, D(B_j)$	Center and mean radius of ball B_j
$D'(B_j)$	Size-weighted mean radius after splitting B_j
$B_{\max}$	Maximum number of granular balls
$n_{\min}$	Minimum number of nodes in each accepted child ball
δ	Minimum relative decrease in mean radius for an accepted split
Local consensus and soft targets	
$\mathbf{w}_j$	Observed class-count vector of training nodes in B_j
U_n	Other training nodes in the ball containing v_n
$s_{\min}$	Minimum number of votes required to use local consensus
$\mathbf{r}_n$	Observed class proportions among the voting nodes in U_n
$i_n^\star$	Class with the unique highest vote count in U_n
$\mathbf{q}_n$	Local-consensus class distribution for node v_n
β	Strength of the adjustment toward the uniquely most-voted class
α_n	Weight of the local consensus in the target for v_n
$\mathbf{e}_i, \mathbb{I}(\cdot)$	One-hot vector for class i; indicator function
$\mathbf{t}_n$	Soft training target for node v_n
$f, \mathbf{p}_n$	MLP classifier and its predicted class distribution for v_n

predicted class distribution for node v_n is

$$\mathbf{p}_n = \mathrm{softmax}\big(f(\mathbf{x}_n)\big). \tag{1}$$

Here, $p_{n,i}$ is the predicted probability that node v_n belongs to class i. After training, only f is retained for inference.

3.2. Label-free representation learning and granular ball construction

GB-LCSS first learns a label-free graph representation and then uses its geometry to partition the nodes into granular balls.

3.2.1. Label-free representation learning

We pretrain GRACE [13] on the node attributes and graph connectivity without class labels and then freeze the encoder. For distance-based grouping, we take the encoder output before the projection head and normalize each node

representation. The resulting GRACE representation matrix is

$$\mathbf{G} = [\mathbf{g}_1, \dots, \mathbf{g}_N]^\top \in \mathbb{R}^{N \times h}, \tag{2}$$

where $\mathbf{g}_n$ is the normalized representation of the n-th node and h is the representation dimension. Let $\mathbf{g}'_n$ denote the encoder output for node v_n before normalization. Its normalized representation is

$$\mathbf{g}_n = \frac{\mathbf{g}'_n}{\max(\|\mathbf{g}'_n\|_2, \varepsilon)}. \tag{3}$$

Here, $\|\cdot\|_2$ denotes the Euclidean norm, and $\varepsilon = 10^{-12}$ is a numerical safeguard. The same row-wise normalization is applied to all nodes to form $\mathbf{G}$. Section 4.3.3 examines alternative representations and the variation across GRACE pretraining seeds.

3.2.2. Granular ball construction

We denote the collection of granular balls constructed from $\mathbf{G}$ by $\mathcal{B} = \{B_1, \dots, B_M\}$, where B_j is the j-th ball, a subset of $\mathcal{V}$, and M is the final number of balls. Each node is assigned to exactly one ball, whose training nodes provide local label evidence. Construction starts with $\mathcal{B} = \{\mathcal{V}\}$ and recursively splits its member balls. To decide whether a split improves the grouping, we first measure how tightly the node representations in a ball are concentrated around its center.

Definition 1. For a nonempty granular ball B_j, each member node $v_n \in B_j$ has the normalized representation $\mathbf{g}_n$. The center $\boldsymbol{\mu}_j$ and mean radius $D(B_j)$ of the ball are

$$\boldsymbol{\mu}_j = \frac{1}{|B_j|} \sum_{v_n \in B_j} \mathbf{g}_n, \qquad D(B_j) = \frac{1}{|B_j|} \sum_{v_n \in B_j} \|\mathbf{g}_n - \boldsymbol{\mu}_j\|_2. \tag{4}$$

The mean radius is the average Euclidean distance from the member representations to the ball center. A smaller value indicates a more compact ball.

Let $n_{\min}$ be a positive integer denoting the minimum number of nodes allowed in each child ball. A ball B_j in the current collection is eligible for splitting only if $|B_j| \geq 2n_{\min}$. Eligible balls are examined in descending order of $D(B_j)$, so that more dispersed balls are considered first. For each candidate ball B_j, 2-means produces two child balls $B_j^{(1)}$ and $B_j^{(2)}$.

Definition 2. The size-weighted post-split dispersion of B_j is

$$D'(B_j) = \frac{|B_j^{(1)}| D(B_j^{(1)}) + |B_j^{(2)}| D(B_j^{(2)})}{|B_j|}. \tag{5}$$

Let $\delta \in (0, 1)$ specify the minimum relative decrease in the mean radius. The split is accepted only if both child balls contain at least $n_{\min}$ nodes and

$$D'(B_j) \le (1 - \delta) D(B_j). \tag{6}$$

Weighting the child radii by their sizes makes $D'(B_j)$ the average distance from the same nodes to their new centers, so Eq. (6) requires this average to decrease by at least a fraction δ. We accept the first candidate split that satisfies both conditions, update the ball collection, and reorder the remaining candidates.

Splitting stops when no valid candidate remains or when the number of balls reaches the maximum permitted number $B_{\max}$, so that $M \le B_{\max}$. Section 4.1.4 gives the experimental values of the partition parameters, and Algorithm 1 summarizes the procedure. Its Boolean flag accepted records whether a split has been accepted in the current pass; if no split is accepted, construction stops.

Algorithm 1. Recursive granular ball partitioning in the frozen representation space

Input: Normalized frozen representation $\mathbf{G}$, node set $\mathcal{V}$, parameters $B_{\max}, n_{\min}, \delta$
Output: Granular ball partition $\mathcal{B}$

$\mathcal{B} \leftarrow \{\mathcal{V}\}$
while $|\mathcal{B}| < B_{\max}$ **do**
  accepted $\leftarrow$ false
  for each $B_j \in \mathcal{B}$ *with* $|B_j| \ge 2n_{\min}$, *in descending order of* $D(B_j)$ **do**
    Apply 2-means to the normalized representations of nodes in B_j to obtain $B_j^{(1)}$ and $B_j^{(2)}$
    if $\min(|B_j^{(1)}|, |B_j^{(2)}|) \ge n_{\min}$ *and* $D'(B_j) \le (1 - \delta) D(B_j)$ **then**
      $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B_j\}) \cup \{B_j^{(1)}, B_j^{(2)}\}$
      accepted $\leftarrow$ true
      break
    end if
  end for
  if accepted = false **then**
    break
  end if
end while
return $\mathcal{B}$

3.3. Local-consensus supervision and MLP training

Given the partition, GB-LCSS constructs each training target by summarizing the observed labels of the other training nodes in the same ball and combining their consensus with the node's own observed label. These targets then supervise MLP training.

3.3.1. Within-ball label consensus

Consider a training node v_n and the granular ball B_j containing it. We construct its target using a leave-one-out vote that excludes v_n, and repeat this calculation for every training node.

Definition 3. The voting set U_n contains the other training nodes in B_j:

$$U_n = (B_j \cap \mathcal{V}_L) \setminus \{v_n\}. \tag{7}$$

Let $s_{\min} \geq 1$ be an integer specifying the minimum number of voting nodes required to use local consensus. When $|U_n| \geq s_{\min}$, the observed labels of the voting nodes define the empirical class distribution $\mathbf{r}_n = (r_{n,1}, \ldots, r_{n,C})^\top$, with

$$r_{n,i} = \frac{1}{|U_n|} \sum_{v_\ell \in U_n} \mathbb{I}(z_\ell = i), \qquad i = 1, \ldots, C. \tag{8}$$

Here, z_ℓ is the observed label of voting node v_ℓ, and $\mathbb{I}(z_\ell = i)$ equals one if this label is i and zero otherwise. The entries of $\mathbf{r}_n$ are nonnegative and sum to one. Excluding v_n prevents its observed label from reinforcing the consensus used to adjust it.

Definition 4. If a single class receives more votes than every other class, its index is

$$i_n^\star = \arg\max_i r_{n,i}. \tag{9}$$

The local-consensus distribution $\mathbf{q}_n$ assigns additional weight to this class through the parameter $\beta \in [0, 1]$:

$$\mathbf{q}_n = \begin{cases} (1-\beta)\mathbf{r}_n + \beta \mathbf{e}_{i_n^\star}, & \text{if a unique highest-vote class exists} \\ \mathbf{r}_n, & \text{if the highest vote count is tied,} \end{cases} \tag{10}$$

Here, $\mathbf{e}_i$ is the one-hot vector with value 1 at class i and 0 elsewhere. The parameter β controls the weight assigned to the one-hot distribution of the uniquely most-voted class.

3.3.2. Consensus-weighted soft targets

The distribution $\mathbf{q}_n$ summarizes the observed labels of the other training nodes. Its weight in the final target is determined by the largest class proportion in the original vote distribution $\mathbf{r}_n$.

Definition 5. When $|U_n| \geq s_{\min}$, the consensus weight α_n is defined by rescaling the largest vote proportion relative to the uniform class probability $1/C$:

$$\alpha_n = \frac{C \max_i r_{n,i} - 1}{C - 1}. \tag{11}$$

Because $\mathbf{r}_n$ is a class distribution, its largest entry lies between $1/C$ and 1. Equation (11) maps this interval onto $[0, 1]$: uniformly distributed votes give $\alpha_n = 0$, and unanimous votes give $\alpha_n = 1$. For fixed votes, changing β leaves α_n unchanged.

Definition 6. The soft target $\mathbf{t}_n$ combines the observed label of v_n with its local consensus:

$$\mathbf{t}_n = \begin{cases} (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n \mathbf{q}_n, & |U_n| \geq s_{\min} \\ \mathbf{e}_{z_n}, & |U_n| < s_{\min}, \end{cases} \tag{12}$$

The target is a convex combination of the observed one-hot label and local consensus, with α_n controlling the consensus contribution. Nodes with fewer than $s_{\min}$ other training nodes in their ball retain their observed one-hot labels.

Equations (7)–(12) use only the observed labels of training nodes; validation and test labels are not used in target construction. For each ball B_j, Algorithm 2 stores the observed class counts of its training nodes in a vector $\mathbf{w}_j$. For each training node $v_n \in B_j$, it subtracts the node's own vote from $\mathbf{w}_j$ and constructs $\mathbf{t}_n$ from the remaining counts.

Algorithm 2. Granular ball soft-target construction

Input: Granular ball partition $\mathcal{B}$, training nodes $\mathcal{V}_L$, observed labels $\{z_n : v_n \in \mathcal{V}_L\}$, number of classes C, parameters $s_{\min}, \beta$
Output: One stored soft target for each node in $\mathcal{V}_L$

Initialize $\mathbf{t}_n \leftarrow \mathbf{e}_{z_n}$ for every training node $v_n \in \mathcal{V}_L$
for each $B_j \in \mathcal{B}$ **do**
  $\mathbf{w}_j \leftarrow \sum_{v_\ell \in B_j \cap \mathcal{V}_L} \mathbf{e}_{z_\ell}$
  for each $v_n \in B_j \cap \mathcal{V}_L$ **do**
    $U_n \leftarrow (B_j \cap \mathcal{V}_L) \setminus \{v_n\}$
    if $|U_n| < s_{\min}$ **then**
      continue
    end if
    $\mathbf{r}_n \leftarrow (\mathbf{w}_j - \mathbf{e}_{z_n})/|U_n|$
    if *one class* $i_n^\star$ *has a strictly larger vote count than every other class* **then**
      $\mathbf{q}_n \leftarrow (1-\beta)\mathbf{r}_n + \beta \mathbf{e}_{i_n^\star}$
    else
      $\mathbf{q}_n \leftarrow \mathbf{r}_n$
    end if
    $\alpha_n \leftarrow (C \max_i r_{n,i} - 1)/(C-1)$
    $\mathbf{t}_n \leftarrow (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n \mathbf{q}_n$
  end for
end for
return Stored targets for all nodes in $\mathcal{V}_L$

3.3.3. MLP training

The two-layer MLP f consists of a hidden linear layer, followed by ReLU, batch normalization, and dropout, and an output layer that produces class logits. The training loss is

$$\mathcal{L} = -\frac{1}{|\mathcal{V}_L|} \sum_{v_n \in \mathcal{V}_L} \sum_{i=1}^{C} t_{n,i} \log p_{n,i}. \tag{13}$$

3.4. Analysis of local-consensus supervision

Under independent symmetric label noise, we bound the local-vote error probability and derive a sufficient condition for a lower expected soft-target error. The partition, the training-node set, and the clean labels are fixed. Let η denote the corruption probability: each label is independently retained with probability $1-\eta$ and otherwise replaced by a uniformly chosen incorrect class.

Fix a training node v_n with clean label y_n and $s_n = |U_n| \geq s_{\min}$ voting nodes. The clean class proportions and the margin of the true class over its closest competitor are

$$a_{n,i} = \frac{1}{s_n} \sum_{v_\ell \in U_n} \mathbb{I}(y_\ell = i), \qquad \Delta_n = a_{n,y_n} - \max_{i \neq y_n} a_{n,i}. \tag{14}$$

Here, $a_{n,i}$ is the fraction of nodes in U_n whose clean label is i, and a_{n,y_n} selects the proportion for the true class of v_n. A positive Δ_n means that this class receives more clean votes than any competing class. We use Pr to denote probability over the independent label corruptions.

Proposition 1. *Suppose that $C \geq 2$, $\Delta_n > 0$, and $0 \leq \eta < (C-1)/C$, and let $\gamma = 1 - C\eta/(C-1) > 0$. Each voting label is independently flipped with probability η to a uniformly chosen incorrect class. Then*

$$\Pr\left(\max_{i \neq y_n} r_{n,i} \geq r_{n,y_n}\right) \leq \min\left\{1, (C-1)\exp\left(-\frac{s_n \gamma^2 \Delta_n^2}{2}\right)\right\}. \tag{15}$$

Proof. Fix a competing class $i \neq y_n$. For each voting node $v_\ell \in U_n$, define $\xi_\ell = \mathbb{I}(z_\ell = y_n) - \mathbb{I}(z_\ell = i)$. These variables are independent and lie in $[-1, 1]$, and their average is $\bar{\xi} = r_{n,y_n} - r_{n,i}$. Under symmetric label corruption,

$$\mathbb{E}[\bar{\xi}] = \gamma(a_{n,y_n} - a_{n,i}) \geq \gamma\Delta_n.$$

Hoeffding's inequality [45] therefore gives

$$\Pr\left(r_{n,i} \geq r_{n,y_n}\right) = \Pr(\bar{\xi} \leq 0) \leq \exp\left(-\frac{s_n \gamma^2 \Delta_n^2}{2}\right).$$

Taking the union bound over the $C-1$ competing classes and capping the bound at one yields Eq. (15).

Symmetric corruption reduces the expected margin over the closest competitor from Δ_n to $\gamma\Delta_n$. For a fixed positive margin and noise rate, the exponential term in the bound decreases as the number of votes increases. However, enlarging a group can also change its class composition and reduce the margin, so both quantities matter for voting reliability. The adjustment in Eq. (10) preserves the most-voted classes, and the same bound therefore applies to $\mathbf{q}_n$. Appendix H.1 in the Supplementary Material gives the detailed expectation calculation.

Proposition 1 bounds the probability that local voting fails to give the true class a unique lead. We next assess the full soft-target distribution through its squared distance from the clean one-hot label. For node v_n, $\mathbf{e}_{z_n}$ is its observed one-hot label. The Brier score of a target vector $\mathbf{b}$ is $\mathrm{BS}_n(\mathbf{b}) = \|\mathbf{b} - \mathbf{e}_{y_n}\|_2^2$. The target $\mathbf{t}_n = (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n \mathbf{q}_n$ satisfies

$$\mathrm{BS}_n(\mathbf{t}_n) = (1-\alpha_n)\,\mathrm{BS}_n(\mathbf{e}_{z_n}) + \alpha_n\,\mathrm{BS}_n(\mathbf{q}_n) - \alpha_n(1-\alpha_n)\|\mathbf{e}_{z_n} - \mathbf{q}_n\|_2^2. \tag{16}$$

This identity expresses the error of the soft target as the weighted sum of the errors of the observed label and the local consensus, minus a nonnegative term determined by their disagreement. The following corollary gives a sufficient condition for an improvement in expectation.

Corollary 1. *For the fixed node v_n with $|U_n| \geq s_{\min}$, assume independent symmetric label corruption with rate η. Fix the partition, the training-node set, the clean labels, and the observed labels of all other training nodes. Then $\mathbf{q}_n$ and α_n are*

fixed, and

$$\mathbb{E}[\mathrm{BS}_n(\mathbf{t}_n)] \le 2\eta(1-\alpha_n) + \alpha_n \mathrm{BS}_n(\mathbf{q}_n), \tag{17}$$

where the expectation is taken over the corruption of node v_n's label. If $\alpha_n > 0$ and $\mathrm{BS}_n(\mathbf{q}_n) < 2\eta$, the soft target has a lower expected squared error than the observed one-hot label.

Because node v_n is excluded from its own vote, $\mathbf{q}_n$ and α_n do not change with its observed label, whose one-hot vector has expected squared error 2η. Taking the expectation of Eq. (16) and omitting its nonpositive final term therefore gives Eq. (17) (Appendix H.2). The condition compares the error of the consensus distribution with the expected error of a noisy annotation.

Across the training set, the change in Brier score combines the error reduction on corrupted labels with any error introduced on correct labels. Section 4.3.1 reports the net reduction, and Appendix I.2 measures the two components separately.

3.5. Graph-teacher distillation extension

The base method trains the MLP on the labeled training nodes. In the extension, the same targets supervise a GCN teacher, whose predictions on all nodes are distilled into an MLP student.

The teacher is a two-layer GCN that takes the original attributes $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$ as inputs. Its class-logit matrix is

$$\mathbf{O} = \mathrm{GCN}(\mathbf{X}, \mathbf{A}) \in \mathbb{R}^{N\times C}. \tag{18}$$

Let $\mathbf{o}_n \in \mathbb{R}^C$ denote the teacher-logit vector for node v_n, whose transpose is the n-th row of $\mathbf{O}$. The teacher loss is

$$\mathcal{L}_1 = -\frac{1}{|\mathcal{V}_L|} \sum_{v_n\in\mathcal{V}_L} \sum_{i=1}^{C} t_{n,i} \log\left[\mathrm{softmax}(\mathbf{o}_n)\right]_i. \tag{19}$$

The teacher therefore uses $\mathbf{G}$ only through the GB-LCSS targets; its inputs are $\mathbf{X}$ and $\mathbf{A}$.

After checkpoint selection, the teacher is frozen. With distillation temperature $\tau > 0$, its prediction for node v_n is

$$\boldsymbol{\pi}_n = \mathrm{softmax}\left(\frac{\mathbf{o}_n}{\tau}\right). \tag{20}$$

Here, $\pi_{n,i}$ is the teacher's probability for class i at node v_n. Increasing τ produces a softer distribution. The resulting distribution is stored as the distillation target of the node.

The MLP student f is trained on all N nodes, with these distributions as its sole distillation targets. Its loss is

$$\mathcal{L}_2 = -\frac{\tau^2}{N} \sum_{n=1}^{N} \sum_{i=1}^{C} \pi_{n,i} \log\left[\mathrm{softmax}\left(\frac{f(\mathbf{x}_n)}{\tau}\right)\right]_i. \tag{21}$$

After distillation, only the MLP student is retained for inference on node attributes.

3.6. Computational costs

Relative to direct MLP training, GB-LCSS adds three training-time stages: graph contrastive pretraining, granular ball partitioning, and soft-target construction. The pretraining cost depends on the encoder and the training schedule; Section 4.4.2 reports the measured cost. Pretraining is shared across downstream runs that use the same frozen representation.

The cost of partitioning is dominated by the 2-means calls. Running 2-means for I iterations on a ball B_j, with a fixed number of initializations, costs $\mathcal{O}(I\,|B_j|\,h)$. The candidate balls examined in one pass of Algorithm 1 are disjoint, so their combined size is at most N. Each pass either accepts one split or terminates the algorithm. There are at most $B_{\max}-1$ successful passes and one final unsuccessful pass, so the total cost of all 2-means calls is bounded by

$$\mathcal{O}(I\,B_{\max}\,N\,h). \tag{22}$$

For fixed I and $B_{\max}$, this bound is linear in both the number of nodes and the representation dimension. Sorting the candidates by dispersion in each pass adds at most $\mathcal{O}(B_{\max}^2 \log B_{\max})$ operations in total. The frozen representation $\mathbf{G}$ requires $\mathcal{O}(Nh)$ space, and the node assignments, ball centers, and per-ball class counts require $\mathcal{O}(N + B_{\max}(h+C))$ space. Given the per-ball class counts, computing the targets for the labeled nodes takes $\mathcal{O}(|\mathcal{V}_L|C)$ time. The implementation initializes and stores a dense $N \times C$ target matrix, which adds $\mathcal{O}(NC)$ time and space.

The classifier is the same two-layer MLP as that used in the baselines. Let F denote its hidden width and T_S the number of training epochs. The loss in Eq. (13) involves only the training nodes, but the full-batch implementation evaluates all N nodes in each epoch, at a total cost of $\mathcal{O}(T_S N(dF + FC))$. At inference, the cost per node is

$$\mathcal{O}(dF + FC). \tag{23}$$

The model parameters also require $\mathcal{O}(dF+FC)$ space. The extension additionally trains a GCN teacher and then trains an MLP student on the teacher's predictions for all N nodes. Its deployed MLP has the same architecture and inference cost as above.

4. Experiments

4.1. Experimental setup

4.1.1. Datasets

The main experiments use five public datasets: the citation graphs Cora and CiteSeer [46], the 70-class citation graph CoraFull, the co-authorship graph Coauthor-CS, and the co-purchase graph Amazon-Photo [47]. Table 2 summarizes their

Table 2
Statistics of the five main datasets.

Dataset	Nodes	Undirected edges	Features	Classes	Training nodes	Edge homophily
Cora	2,708	5,278	1,433	7	541	0.810
CiteSeer	3,327	4,552	3,703	6	665	0.736
CoraFull	19,793	63,421	8,710	70	3,958	0.567
Coauthor-CS	18,333	81,894	6,805	15	3,666	0.808
Amazon-Photo	7,650	119,081	745	8	1,530	0.827

statistics. Edges are counted as unique undirected pairs after self-loop removal, training-node counts refer to the 20% split, and edge homophily is the proportion of edges that connect nodes of the same class. Six additional datasets are evaluated under uniform 0.4 noise (Appendix G).

4.1.2. Data splits and label noise

All comparisons follow the fixed-graph transductive protocol of Section 3.1. For each seed from 0 to 4, we use the NoisyGL percentage-split routine to randomly sample training, validation, and test nodes from the full node set at ratios of 20%, 10%, and 70%, respectively. Noise is injected only into the training labels, and all methods share the same split and noisy labels for each dataset and seed. Clean validation labels are used for checkpoint selection, and test labels for final evaluation. Appendix F gives the exact subset sizes and the sampling procedure, and Appendix C examines sensitivity to corrupted validation labels.

Under uniform noise, each training label is retained with probability $1-\eta$ and is replaced by each of the other $C-1$ classes with probability $\eta/(C-1)$. We evaluate $\eta \in \{0.2, 0.4, 0.6\}$. Pair noise uses $\eta = 0.4$: it changes class $i \in \{1, \dots, C\}$ to $((i-2) \bmod C) + 1$ with probability η and leaves it unchanged with probability $1-\eta$. Instance-dependent noise is generated with the NoisyGL generator, using a configured mean flip rate of 0.4 and norm_std $= 0.1$.

4.1.3. Compared methods

The first comparison group comprises seven noisy-label baselines: MLP+CE, Small-loss, Co-teaching, JoCoR, SCE, ELR, and LogitClip. They use the same deployed two-layer MLP and the same labeled training nodes as GB-LCSS. GB-LCSS uses graph information during training and requires no noise-model parameters. Small-loss, Co-teaching, and JoCoR use the true noise rate to set their forget-rate schedules. Small-loss selects samples by their per-sample cross-entropy. Co-teaching and JoCoR train two independently initialized MLPs but evaluate a single network without ensembling.

The second group comprises graph-assisted MLP methods: PosteL [38], ALS [39], SALS [40], Graph-MLP [2], and GSDN [30]. All of them use graph information only during training in our protocol. PosteL uses the official target-construction code with iterative pseudo-labeling disabled. Graph-MLP is adapted to the common MLP, whereas ALS, SALS, and GSDN are implemented as mechanism adaptations under the common protocol. Appendix F identifies the retained components and parameter sources for each method.

The third group comprises graph-to-MLP distillation methods: a GCN(X)+CE response-distillation baseline based on GLNN [3], FF-G2M [5], KRD [32], VQGraph [33], and a GCN teacher trained on GB-LCSS targets (Section 3.5). FF-G2M is reimplemented with the common backbones and its low- and high-frequency objectives. The mechanism adaptations

Table 3
Test accuracy of noisy-label methods with the same deployed MLP under uniform 0.4 noise.

Method	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
MLP+CE	44.24±2.62	43.50±3.02	31.98±0.98	88.42±0.62	73.90±1.80
Small-loss	51.95±1.97	49.70±3.07	33.04±0.50	89.06±0.88	77.25±1.62
Co-teaching	46.83±1.78	46.24±2.89	34.39±0.24	88.80±0.97	78.21±1.00
JoCoR	51.72±1.30	49.91±3.37	35.70±0.32	89.38±0.61	79.57±0.53
SCE	43.60±1.86	42.09±3.47	29.29±0.45	88.37±0.53	76.08±3.11
ELR	44.28±1.38	43.17±3.95	32.14±0.67	88.29±0.73	73.93±2.12
LogitClip	47.50±1.50	46.57±3.52	34.69±1.17	88.40±0.21	74.98±2.38
GB-LCSS	**57.49±0.76**	**55.24±2.04**	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

of KRD and VQGraph retain reliability-based knowledge selection and soft-codebook distillation, respectively. For each seed, the teacher and student are retrained for every route, using the same MLP configuration for all students.

4.1.4. Implementation details

Datasets are loaded through the NoisyGL interface to PyTorch Geometric. Graph contrastive pretraining uses row-normalized attributes, whereas downstream models use the attributes supplied by NoisyGL. Graph-convolutional models use the supplied adjacency, with normalization and self-loops handled within their graph-convolution layers.

The noisy-label baselines and GB-LCSS use a two-layer MLP with a hidden width of 64 and a dropout rate of 0.5. We train for 200 epochs using Adam with a learning rate of 0.01 and a weight decay of 5×10^{-4}, and we select the checkpoint with the highest validation accuracy. Teacher checkpoints are selected with the same criterion.

By default, GB-LCSS uses $B_{\max} = 64$, $n_{\min} = 32$, $\delta = 0.05$, $s_{\min} = 2$, and $\beta = 0.5$. Recursive 2-means uses Euclidean distance, k-means++ initialization [48], and 10 initializations per candidate split, with randomization controlled by the downstream seed. The distillation extension uses a temperature of $\tau = 1$. GRACE configurations and all other parameter settings, with their sources, are given in Appendix F.

4.1.5. Evaluation protocol

Test-node classification accuracy is reported in percent as the mean ± sample standard deviation over five paired seeds. Bold marks the highest mean within each dataset and noise setting; in Table 4, underlining marks the second-highest mean. Each dataset uses one fixed normalized pretrained representation $\mathbf{G}$. Section 4.3.3 separately examines variation across GRACE pretraining seeds.

Same-seed accuracy differences are assessed with two-sided paired t-tests and 95% confidence intervals. Holm correction [49] is applied separately to each family of 25 tests, with one family for each reference method: random size-matched grouping, MLP+CE, and JoCoR. Appendix B reports the paired comparisons, confidence intervals, and Holm-adjusted p-values.

4.2. Comparative results

4.2.1. Comparison with noisy-label baselines

GB-LCSS attains the highest mean accuracy on all five datasets under uniform 0.4, exceeding the strongest baseline on each dataset by 1.33–5.54 percentage points (Table 3). Across all five noise settings, it achieves the highest mean in 24 of the 25 settings, with average gains of 7.74 points over MLP+CE and 3.46 points over JoCoR, the strongest baseline on average across the 25 settings (Fig. 2). The only exception is Coauthor-CS under uniform 0.2, where JoCoR is higher by 0.27 points. Appendix A gives the complete accuracy results, and Appendix B reports the paired statistical comparisons.

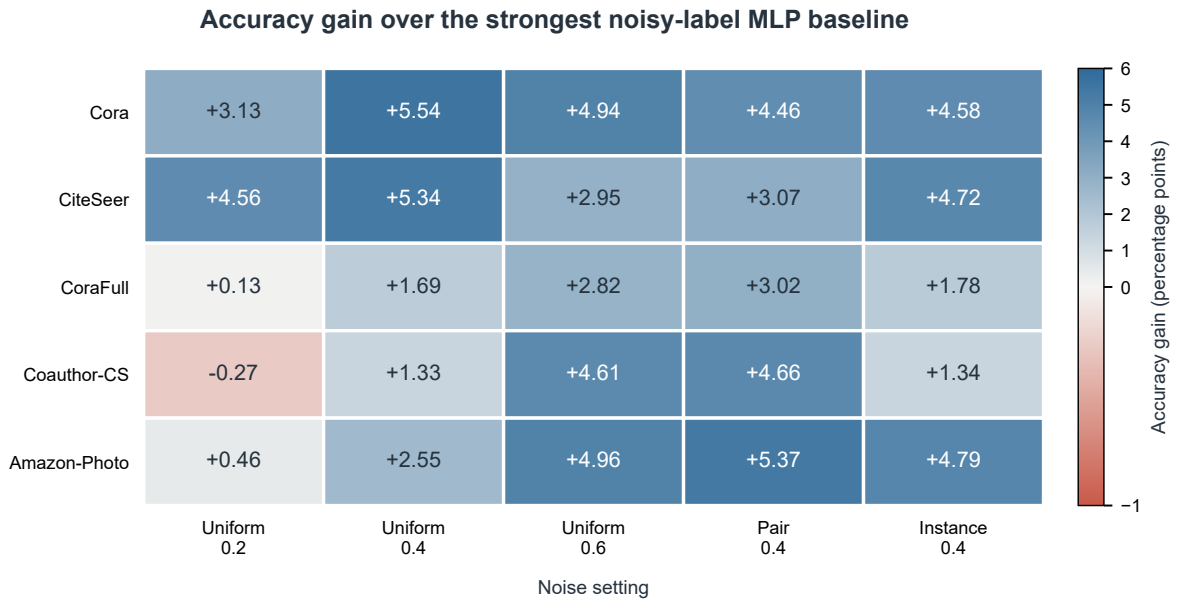


Figure 2: Accuracy gain of GB-LCSS over the strongest noisy-label baseline with the same deployed MLP. Each cell shows, in percentage points, the five-seed mean accuracy of GB-LCSS minus the highest mean among the seven baselines for the same dataset and noise setting. Blue and red denote positive and negative gains, respectively.

Fig. 3 shows accuracy as a function of the uniform noise rate. As the rate increases from 0.2 to 0.6, GB-LCSS loses less accuracy than MLP+CE and JoCoR on all datasets except CiteSeer.

On six additional datasets under uniform 0.4 noise, GB-LCSS exceeds random size-matched grouping on all six and the strongest noisy-label baseline on three (Appendix G).

4.2.2. Comparison with graph-assisted MLP methods

Table 4 compares the graph-assisted MLP methods, which use the graph either to construct soft targets or to shape the representation of the MLP.

Across the 25 settings, the average accuracy gains over PosteL, ALS, and SALS are 5.69, 7.71, and 4.69 percentage points, respectively. GB-LCSS achieves higher mean accuracy than PosteL and SALS in 24 settings each, and than ALS in all 25 settings (Appendix E). These methods use graph-defined neighborhoods for target construction, whereas GB-LCSS uses regions of a label-free representation and leave-one-out voting.

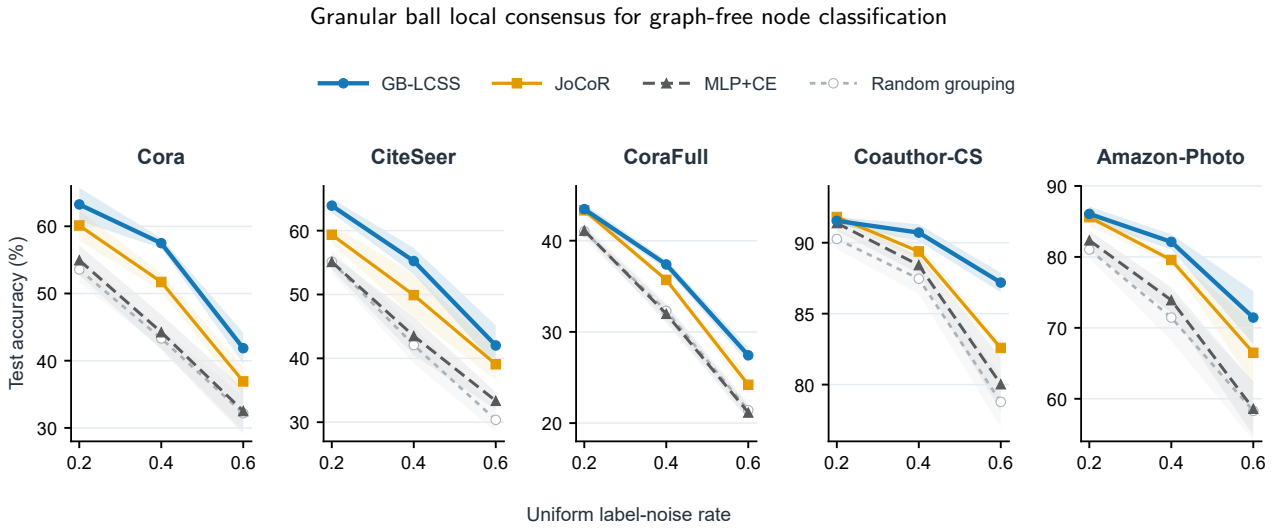


Figure 3: Test accuracy under uniform label noise. Lines show the means over five paired seeds, and shaded regions show the sample standard deviations. Panels use different y-axis scales. "Random grouping" denotes random size-matched grouping, which preserves the number and sizes of the granular balls.

Table 4
Test accuracy of methods that use the graph during MLP training, under uniform 0.4 noise.

Method	Use of graph structure in training	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
PosteL	Posterior soft labels	47.09±1.81	45.84±2.48	36.51±1.01	84.74±1.51	80.59±0.94
ALS	Neighborhood label smoothing	45.20±1.24	43.77±2.76	31.91±1.05	88.55±0.86	73.41±2.75
SALS	Structure-aware label smoothing	48.27±2.50	47.20±1.69	37.06±0.78	88.85±0.82	77.86±0.79
Graph-MLP	Multi-hop neighborhood-contrastive objective	**63.23±3.07**	**57.23±1.85**	37.05±0.91	88.95±0.43	77.05±1.24
GSDN	Neighborhood self-distillation	55.97±4.29	50.36±3.39	35.80±0.66	86.55±0.78	73.31±2.00
GB-LCSS	Label consensus within granular balls	57.49±0.76	55.24±2.04	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

Graph-MLP and GSDN use the graph to shape the representation of the MLP and leave the labels unchanged. Under uniform 0.4, GB-LCSS has a higher mean than Graph-MLP on CoraFull, Coauthor-CS, and Amazon-Photo, by 0.34, 1.76, and 5.07 points, respectively, whereas Graph-MLP has the higher mean on Cora and CiteSeer, by 5.74 and 1.99 points. GB-LCSS exceeds GSDN on all five datasets. Graph-MLP and GB-LCSS thus use the training graph in different ways, shaping the learned representation and refining the supervision targets, respectively, and neither is uniformly better across datasets.

4.3. Ablation and mechanism analysis

4.3.1. Effect of local-consensus targets

We first examine how the choice of nodes within each ball affects the benefit of local-consensus supervision. Random size-matched grouping shuffles node membership while preserving the ball sizes, and it leaves the target rule unchanged. Uniform and prior smoothing use a fixed coefficient of 0.1 to mix each observed one-hot label with a uniform class distribution or the observed training-label frequencies, respectively. These controls test whether ordinary label smoothing

Table 5
Target-construction ablations under uniform 0.4 noise.

Dataset	GB-LCSS	Own vote included	Random size-matched grouping	Uniform smoothing	Prior smoothing	Gain from excluding own vote
Cora	**57.49±0.76**	56.28±1.54	43.31±1.40	44.69±2.90	44.89±2.38	+1.21
CiteSeer	**55.24±2.04**	53.52±2.71	42.08±2.88	43.69±3.31	43.43±2.99	+1.72
CoraFull	**37.39±0.51**	37.30±0.57	32.30±0.85	32.59±1.12	32.63±0.84	+0.09
Coauthor-CS	**90.71±0.59**	90.62±0.47	87.46±1.07	88.25±0.80	88.04±0.73	+0.09
Amazon-Photo	**82.12±1.08**	81.33±0.70	71.45±3.29	74.18±1.72	74.72±1.44	+0.79

provides similar gains. A further variant includes the current node's own label in the vote under the same tie-handling rule. Table 5 reports these comparisons under uniform 0.4.

Random size-matched grouping lowers accuracy in all 25 settings, by 8.69 percentage points on average, to approximately the level of MLP+CE. Uniform and prior smoothing recover little of this gain. These controls support the importance of which nodes contribute to the local consensus, beyond target softening alone. Excluding the current node's vote raises mean accuracy on all five datasets under uniform 0.4, as well as on every dataset when averaged over the five noise settings. With clean training labels, targets change only in balls whose training nodes belong to more than one class. Accuracy differs from that of MLP+CE by -1.75 to $+2.00$ percentage points (Appendix D).

We then assess the training targets themselves, independently of the classifier. Fig. 4 reports their multiclass Brier score [50], averaged over the labeled training nodes and computed against the clean training labels, which are used only for evaluation:

$$\frac{1}{|\mathcal{V}_L|}\sum_{v_n\in\mathcal{V}_L}\|\mathbf{t}_n-\mathbf{e}_{y_n}\|_2^2.$$

The score of the observed labels is computed by replacing $\mathbf{t}_n$ with $\mathbf{e}_{z_n}$. All 25 points, each averaged over five seeds, lie below the equality line: the GB-LCSS targets reduce the score by 0.06–0.51 relative to the observed one-hot targets, with the largest reductions on Coauthor-CS and Amazon-Photo.

Using Eq. (H.5), Appendix I.2 separates the net Brier-score reduction under uniform 0.4 noise into error removed on corrupted labels and error introduced on correct labels. The reduction exceeds the increase in all 25 dataset–seed runs, by a factor of 6.7–18.1 in the five-seed means.

4.3.2. Sensitivity to the grouping rule

This experiment tests whether local-consensus supervision depends on the specific partitioning procedure. We keep the frozen representation and Eqs. (7)–(12) unchanged and replace the granular ball partition with kNN neighborhoods or a k-means partition. For each seed, kNN excludes the query node and matches its neighborhood size to the mean number of training votes per ball; k-means uses the same number of groups as the granular ball partition and likewise excludes the

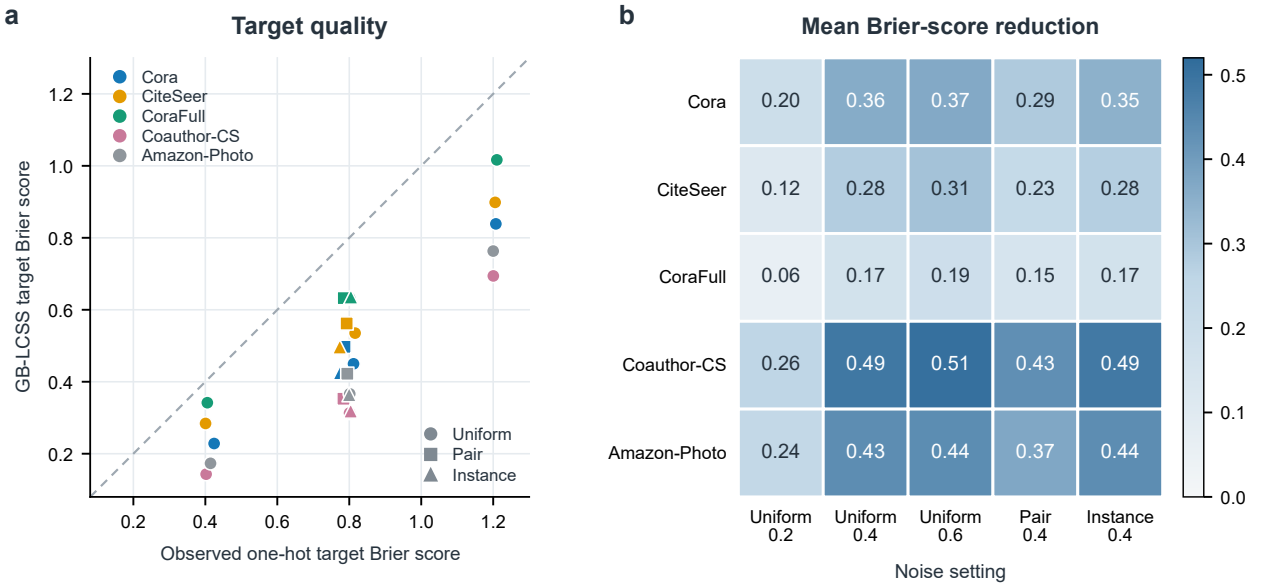


Figure 4: Brier scores of the GB-LCSS targets and the observed one-hot targets, computed against the clean training labels. (a) The horizontal and vertical axes show the multiclass Brier scores of the observed one-hot targets and the GB-LCSS targets, respectively. Each marker is the five-seed mean for one dataset–noise setting; colors identify datasets, and circles, squares, and triangles denote uniform, pair, and instance-dependent noise, respectively. The dashed line denotes equality. (b) Each cell shows the observed-target score minus the GB-LCSS target score; positive values indicate a lower squared probability error for the soft targets.

Table 6
Sensitivity to the grouping rule under uniform 0.4 noise.

Dataset	Granular ball	kNN	kNN − GB	k-means	k-means − GB
Cora	**57.49±0.76**	57.21±1.19	−0.27	56.58±1.23	−0.91
CiteSeer	55.24±2.04	56.17±3.24	+0.94	**56.18±1.78**	+0.94
CoraFull	37.39±0.51	**40.50±0.51**	+3.11	39.26±0.48	+1.87
Coauthor-CS	90.71±0.59	90.83±0.57	+0.12	**91.01±0.64**	+0.29
Amazon-Photo	82.12±1.08	**82.23±1.20**	+0.11	81.68±0.66	−0.44

GB denotes granular ball grouping. Differences are in percentage points.

current node's own vote. Table 6 reports the comparison under uniform 0.4, with differences defined as the accuracy of the alternative grouping minus that of granular ball grouping, in percentage points.

The three grouping rules yield mean accuracies within one percentage point of one another on four of the five datasets (Table 6). All three exceed random size-matched grouping (Tables 5 and 6). These results show that local-consensus targets can also be constructed with other grouping rules. We use granular balls because geometric splitting adapts the partition under explicit size and ball-count constraints, while each ball provides shared class counts for constructing its members' targets. Section 4.4.2 examines the construction time and storage requirements of this design.

4.3.3. Effect of the representation space

To examine how the representation affects grouping and target construction, Table 7 compares GRACE, raw node attributes $\mathbf{X}$, and two-step propagated attributes $\mathbf{A}^2\mathbf{X}$ on the five datasets under uniform 0.4 noise, using five paired seeds, fixed splitting parameters, and the main training protocol. Ball counts are reported as ranges across these seeds. For $\mathbf{A}^2\mathbf{X}$,

Table 7
Effect of the representation used for partitioning under uniform 0.4 noise.

Dataset	Accuracy			Number of balls		
	GRACE	Raw $\mathbf{X}$	$\mathbf{A}^2\mathbf{X}$	GRACE	Raw $\mathbf{X}$	$\mathbf{A}^2\mathbf{X}$
Cora	**57.49±0.76**	44.37±2.31	44.37±2.31	52–53	1	1
CiteSeer	**55.24±2.04**	42.90±2.50	42.90±2.50	63–64	1	1
CoraFull	**37.39±0.51**	31.86±1.05	31.86±1.05	64	1	1
Coauthor-CS	**90.71±0.59**	87.63±1.10	89.20±0.67	64	1	50–59
Amazon-Photo	**82.12±1.08**	73.37±1.13	81.13±1.64	64	1	64

the attributes are multiplied twice by the original adjacency matrix, without added self-loops or normalization, and the resulting vectors are normalized as in Eq. (3).

With raw $\mathbf{X}$, no split satisfies Eq. (6) in any of the 25 dataset–seed runs, so all nodes remain in one ball and each node draws its consensus from all other training nodes. The $\mathbf{A}^2\mathbf{X}$ representation likewise produces a single ball on Cora, CiteSeer, and CoraFull. It produces 50–59 and 64 balls on Coauthor-CS and Amazon-Photo, respectively, where its accuracy exceeds that of raw $\mathbf{X}$. GRACE yields a multi-ball partition on every dataset and the highest mean accuracy among the three representations, exceeding $\mathbf{A}^2\mathbf{X}$ by 1.51 points on Coauthor-CS and 0.99 points on Amazon-Photo.

On Cora, CiteSeer, and CoraFull, we train three GRACE representations with different pretraining seeds and evaluate each representation using three downstream seeds. Partitions obtained from different pretraining seeds differ substantially, with adjusted Rand indices [51] between 0.247 and 0.365. After averaging over downstream seeds, the standard deviations of accuracy across pretraining seeds are 0.13, 0.45, and 0.60 percentage points, respectively, and GB-LCSS exceeds MLP+CE for every pretraining seed. Thus, although the node assignments change substantially across pretraining seeds, downstream accuracy changes little.

4.3.4. Sensitivity to construction and target parameters

Appendix I.1 varies one grouping or voting parameter at a time on the five datasets under uniform 0.4 noise, with fixed representations and five paired seeds. The largest losses occur when a higher splitting threshold leaves all nodes in one ball or when a higher voting threshold sharply reduces the number of nodes receiving consensus supervision. The remaining tested settings stay within three percentage points of the default. A larger ball budget improves accuracy on CoraFull, and assigning more weight to the most-voted class improves accuracy on four datasets.

4.4. Graph-teacher extension and computational cost

4.4.1. Graph-teacher distillation

We next examine whether training a graph teacher with GB-LCSS targets also improves its distilled MLP student (Section 3.5). Table 8 compares direct MLP training with five distillation routes on the five datasets under uniform 0.4 and 0.6. The first row trains the MLP on the labeled nodes only, whereas the remaining rows use teacher predictions with method-specific objectives and node selection; all routes deploy the same two-layer MLP. The GCN teacher trained on GB-LCSS targets differs from the GCN(X)+CE baseline only in using the soft target of Eq. (12) in place of the observed one-hot label; the other routes are described in Section 4.1.3 and Appendix F.

Table 8
Test accuracy of deployed MLPs under direct training and graph-to-MLP distillation.

(a) Uniform 0.4

Training route	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
GB-LCSS, direct MLP training	57.49±0.76	55.24±2.04	37.39±0.51	90.71±0.59	82.12±1.08
GCN(X)+CE teacher	77.95±2.62	66.09±1.38	**58.74±0.82**	93.20±0.09	89.86±0.77
FF-G2M	77.65±2.97	65.80±0.49	56.58±0.93	91.58±0.24	84.82±0.84
KRD	71.77±2.20	64.96±1.03	47.12±0.95	91.39±0.49	85.17±0.94
VQGraph	78.89±2.34	65.64±1.72	58.52±0.77	93.25±0.09	**90.54±0.85**
GCN teacher trained on GB-LCSS targets	**81.35±1.05**	**69.27±1.57**	57.29±0.24	**93.83±0.21**	90.40±0.77

(b) Uniform 0.6

Training route	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
GB-LCSS, direct MLP training	41.86±2.30	42.02±3.14	27.43±0.89	87.19±0.69	71.45±3.75
GCN(X)+CE teacher	65.50±3.28	53.30±4.42	51.63±2.02	91.13±0.58	84.80±1.48
FF-G2M	63.63±2.16	52.32±4.20	45.78±2.90	86.50±1.01	73.08±2.82
KRD	61.28±1.17	54.17±3.14	41.61±2.43	89.80±0.29	80.30±0.51
VQGraph	65.75±2.72	53.34±4.47	52.01±0.65	90.71±1.24	84.77±2.19
GCN teacher trained on GB-LCSS targets	**71.34±3.91**	**58.93±4.62**	**52.32±1.26**	**92.40±0.65**	**86.81±1.94**

Training the GCN teacher with GB-LCSS targets instead of the observed labels raises the mean accuracy of the distilled student in nine of the ten settings; the only exception is CoraFull under uniform 0.4. Target refinement can therefore improve the supervision transferred through a graph teacher. Among the five distillation routes, this student attains the highest mean accuracy in eight of the ten settings; on CoraFull and Amazon-Photo under uniform 0.4, GCN(X)+CE and VQGraph are higher by 1.45 and 0.14 points, respectively. The distilled student exceeds direct MLP training in all ten settings, by 3.12–23.86 points under uniform 0.4 and 5.21–29.48 points under uniform 0.6. The extension adds a GCN teacher and distillation over all nodes during training, while retaining the same MLP architecture at inference.

4.4.2. Computational cost

GB-LCSS and MLP+CE deploy the same MLP, so we measure the additional training-time costs. Timing and GPU-memory measurements were obtained on a Windows 10 (build 19045) workstation with an AMD Radeon RX 7700 XT GPU, using Python 3.12.10, PyTorch 2.9.1+rocm7.2.1, and PyTorch Geometric 2.7.0. GRACE pretraining takes 0.07, 0.13, and 74.48 minutes on Cora, CiteSeer, and CoraFull, respectively, with peak allocated GPU memory of 0.40, 0.67, and 12.73 GiB. Times are averaged over three pretraining seeds, and memory is the largest peak across these runs reported by `torch.cuda.max_memory_allocated()`. Each frozen representation is computed once per dataset and reused across all noise settings and downstream seeds.

Fig. 5 compares the construction time and storage of the three grouping rules. The granular ball partition stores one class-count vector per ball, whereas exact kNN stores one neighbor list per training node. In the seed-0 measurements, granular ball grouping requires less storage than exact kNN on CoraFull, Coauthor-CS, and Amazon-Photo, with the

largest differences on the two largest graphs. Its construction time is also the shortest among the three grouping rules on three of the five datasets.

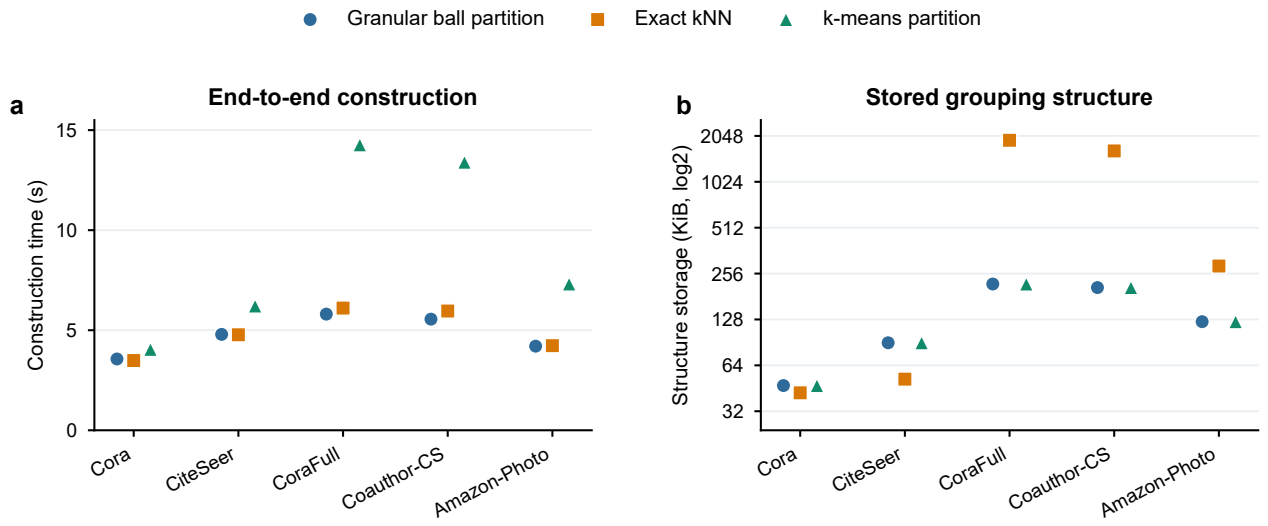


Figure 5: Construction cost of granular ball, exact-kNN, and k-means grouping. (a) End-to-end construction time. (b) Storage of the grouping structure. Both panels show seed-0 measurements.

4.5. Discussion

When the other training labels in a ball provide consistent evidence for the underlying class, their consensus can offset an isolated annotation error. The label-free representation determines which nodes contribute this evidence, and the random-grouping and label-smoothing controls support the importance of this choice beyond target softening alone (Table 5). Leave-one-out voting keeps the current annotation out of its own consensus estimate, a property used in the conditional expected-error analysis of Corollary 1. Under uniform 0.4 noise, the reduction in Brier error on corrupted labels exceeds the error introduced on correctly labeled nodes (Appendix I.2).

The usefulness of local evidence depends on both its class composition and the number of available votes. Proposition 1 relates these quantities to voting reliability under independent symmetric noise. Agreement alone is insufficient when the local labels favor an incorrect class, since the consensus can then move the target away from the true class. The representation and parameter studies show lower accuracy when local groups fail to form or voting support becomes sparse. Under uniform 0.4 noise, CoraFull has both the lowest proportion of training nodes with a positive clean-vote margin (36.13%) and the smallest net reduction in target Brier score among the five datasets (Appendix I.2). With clean training labels, refinement raises mean accuracy on one dataset and lowers it on four, with all changes within 2.00 percentage points (Appendix D). On the six additional datasets, GB-LCSS exceeds the strongest noisy-label baseline on three (Appendix G). These results indicate that the benefit varies across datasets and supervision conditions.

GB-LCSS, Graph-MLP, and GLNN share an attribute-only deployment interface but use the training graph differently. Graph-MLP shapes the representation through a neighborhood-contrastive objective, GLNN transfers predictions from a GNN teacher, and GB-LCSS refines supervision targets. GB-LCSS and Graph-MLP have dataset-dependent advantages

(Table 4). The distillation experiment shows that training the same GCN teacher with GB-LCSS targets can also benefit its MLP student (Table 8). These targets can therefore support both direct MLP training and graph-to-MLP distillation.

Granular balls provide a practical organization for local-consensus supervision. Each node belongs to one ball, whose class counts are shared when constructing targets for its training nodes. Geometric splitting adapts the partition to the representation while limiting the number of balls and enforcing a minimum size for each child ball. Granular ball grouping requires less storage than exact kNN on the three larger graphs and has the shortest construction time on three datasets (Fig. 5). These results support its use as a grouping mechanism that combines competitive accuracy, shared regional statistics, and controlled construction costs.

5. Conclusion

We presented GB-LCSS for learning from noisy training labels when the deployed classifier receives only node attributes. The method organizes nodes into granular balls in a frozen, label-free graph representation and combines each observed label with the consensus of the other training nodes in its ball. Our analysis relates local-vote reliability to the number of available votes and their clean-class margin under symmetric noise, decomposes the squared error of the soft target, and gives a sufficient condition for it to have lower expected error than the observed label.

Across five datasets and five noise settings, GB-LCSS achieves the highest five-seed mean accuracy in 24 of the 25 settings, compared with seven noisy-label baselines that use the same deployed MLP architecture. Its soft targets have lower mean Brier scores than the observed one-hot labels in all 25 settings. The controlled comparisons support the importance of which training nodes contribute to the local consensus, with granular balls providing a compact way to share label statistics. The distillation experiments further show that these targets can improve the MLP student when they are used to train its graph teacher.

The present evaluation focuses on synthetic label noise in a fixed-graph transductive setting, with clean validation labels used for model selection. Future work will test the method on naturally occurring annotation errors and develop model-selection procedures for imperfect validation labels. Another direction is to evaluate generalization to newly arriving nodes and to investigate how the representations and granular ball partitions can be updated as the training graph grows.

Acknowledgements

This work was supported by the Yangtze River Delta Social Assistance Public Service Platform (B801065001FY050) and the National Natural Science Foundation of China (Nos. 62276136, 62206129, and 62375133).

References

[1] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2017.

[2] Y. Hu, H. You, Z. Wang, Z. Wang, E. Zhou, Y. Gao, Graph-MLP: Node classification without message passing in graph (2021). arXiv:2106.04051.

[3] S. Zhang, Y. Liu, Y. Sun, N. Shah, Graph-less neural networks: Teaching old MLPs new tricks via distillation, in: International Conference on Learning Representations, 2022.

[4] Z. Wang, Z. Zhang, C. Zhang, Y. Ye, Training MLPs on graphs without supervision, in: Proceedings of the ACM International Conference on Web Search and Data Mining, 2025, pp. 697–706.

[5] L. Wu, H. Lin, Y. Huang, T. Fan, S. Z. Li, Extracting low-/high-frequency knowledge from graph neural networks and injecting it into MLPs: An effective GNN-to-MLP distillation framework, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 10351–10360.

[6] S. Liu, J. Niles-Weed, N. Razavian, C. Fernandez-Granda, Early-learning regularization prevents memorization of noisy labels, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 20331–20342.

[7] Z. Wang, D. Sun, S. Zhou, H. Wang, J. Fan, L. Huang, J. Bu, NoisyGL: A comprehensive benchmark for graph neural networks under label noise, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, pp. 38142–38170.

[8] E. Dai, C. Aggarwal, S. Wang, NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs, in: Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2021, pp. 227–236.

[9] S. Qian, H. Ying, R. Hu, J. Zhou, J. Chen, D. Z. Chen, J. Wu, Robust training of graph neural networks via noise governance, in: Proceedings of the ACM International Conference on Web Search and Data Mining, 2023, pp. 607–615.

[10] Y. Wu, J. Yao, X. Xia, J. Yu, R. Wang, B. Han, T. Liu, Mitigating label noise on graphs via topological sample selection, in: Proceedings of the International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, 2024, pp. 53944–53972.

[11] A. Iscen, J. Valmadre, A. Arnab, C. Schmid, Learning with neighbor consistency for noisy labels, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 4672–4681.

[12] D. Bahri, H. Jiang, M. Gupta, Deep k-NN for noisy labels, in: Proceedings of the International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 540–550.

[13] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, in: ICML Workshop on Graph Representation Learning and Beyond, 2020.

[14] S. Xia, Y. Liu, X. Ding, G. Wang, H. Yu, Y. Luo, Granular ball computing classifiers for efficient, scalable and robust learning, Information Sciences 483 (2019) 136–152.

[15] S. Xia, S. Zheng, G. Wang, X. Gao, B. Wang, Granular ball sampling for noisy label classification or imbalanced classification, IEEE Transactions on Neural Networks and Learning Systems 34 (4) (2023) 2144–2155.

[16] S. Xia, X. Dai, G. Wang, X. Gao, E. Giem, An efficient and adaptive granular-ball generation method in classification problem, IEEE Transactions on Neural Networks and Learning Systems 35 (4) (2024) 5319–5331.

[17] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. W. Tsang, M. Sugiyama, Co-teaching: Robust training of deep neural networks with extremely noisy labels, in: Advances in Neural Information Processing Systems, Vol. 31, 2018, pp. 8527–8537.

[18] H. Wei, L. Feng, X. Chen, B. An, Combating noisy labels by agreement: A joint training method with co-regularization, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 13726–13735.

[19] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, J. Bailey, Symmetric cross entropy for robust learning with noisy labels, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 322–330.

[20] H. Wei, H. Zhuang, R. Xie, L. Feng, G. Niu, B. An, Y. Li, Mitigating memorization of noisy labels by clipping the model prediction, in: Proceedings of the International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 36868–36886.

[21] Y. Li, J. Yin, L. Chen, Unified robust training for graph neural networks against label noise, in: Advances in Knowledge Discovery and Data Mining, Vol. 12712 of Lecture Notes in Computer Science, 2021, pp. 528–540.

[22] K. Li, J. Sun, J. Lou, Z. Feng, H. Zhou, C. Wu, G. Xue, W. Zhao, J. Li, Leveraging peer-informed label consistency for robust graph neural networks with noisy labels, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2025, pp. 5598–5606.

[23] S. Li, B. Zhang, J. Song, G. Ruan, C. Wang, J. Xie, Graph neural networks with coarse- and fine-grained division for mitigating label noise and sparsity, Neural Networks 187 (2025) 107338.

[24] K. Ding, X. Ma, Y. Liu, S. Pan, Divide and denoise: Empowering simple models for robust semi-supervised node classification against label noise, in: Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 574–584.

[25] Y. Zhu, L. Feng, Z. Deng, Y. Chen, R. Amor, M. Witbrock, Robust node classification on graph data with graph and label noise, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 17220–17227.

[26] Q. Li, X. Li, D. Li, J. Wang, Prototype-guided supervision for graph learning with noisy and sparse labels, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 23105–23113.

[27] B. Hu, G. Wen, Y. Huang, Y. Liu, X. Zhu, Structure-aware multi-teacher distillation for noise node classification, Information Processing & Management 63 (7) (2026) 104877.

[28] Z. Liu, H. Yu, X. Luo, Mitigating label noise in graph learning with information bottleneck-guided aggregation and adversarial consistency, Pattern Recognition 180 (2026) 114271.

[29] S. Li, B. Zhang, M. Cao, J. Song, X. Cheng, C. Wang, HALF: A homophily-aware loss fusion for robust learning under label noise in heterophilic graphs, Information Sciences 754 (2026) 123714.

[30] L. Wu, J. Xia, H. Lin, Z. Gao, Z. Liu, G. Zhao, S. Z. Li, Teaching yourself: Graph self-distillation on neighborhood for node classification (2022). arXiv:2210.02097.

[31] J. Chen, M. Bai, S. Chen, J. Gao, J. Zhang, J. Pu, SA-MLP: Distilling graph knowledge from GNNs into structure-aware MLP, Transactions on Machine Learning Research (2024).

[32] L. Wu, H. Lin, Y. Huang, S. Z. Li, Quantifying the knowledge in GNNs for reliable distillation into MLPs, in: Proceedings of the International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 37571–37581.

[33] L. Yang, Y. Tian, M. Xu, Z. Liu, S. Hong, W. Qu, W. Zhang, B. Cui, M. Zhang, J. Leskovec, VQGraph: Rethinking graph representation space for bridging GNNs and MLPs, in: International Conference on Learning Representations, 2024.

[34] T. Wu, Z. Zhao, J. Wang, X. Bai, L. Wang, N. Wong, Y. Yang, Edge-free but structure-aware: Prototype-guided knowledge distillation from GNNs to MLPs, in: Proceedings of the International Conference on Computational Linguistics, 2025, pp. 5656–5667.

[35] Z. Zhou, Z. Ding, J. Shi, L. Qing, S. Shen, TINED: GNNs-to-MLPs by teacher injection and Dirichlet energy distillation, in: Proceedings of the International Conference on Machine Learning, Vol. 267 of Proceedings of Machine Learning Research, 2025, pp. 78616–78632.

[36] H. Pal, S. Malik, R. Patel, A. Malhotra, SALE-MLP: Structure aware latent embeddings for GNN to graph-free MLP distillation, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2025, pp. 6003–6011.

[37] Z. Yu, M. Lin, W. Li, Z. Yin, S. Xiao, S. Lu, Demystifying GNN-to-MLP knowledge transfer: Theoretical grounding and dual-stream distillation method, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 28005–28013.

[38] J. Heo, M. Park, D. Kim, Posterior label smoothing for node classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 21708–21716.

[39] K. Zhou, S.-H. Choi, Z. Liu, N. Liu, F. Yang, R. Chen, L. Li, X. Hu, Adaptive label smoothing to regularize large-scale graph training, in: Proceedings of the 2023 SIAM International Conference on Data Mining, 2023, pp. 55–63.

[40] Y. Wang, Y. Cai, Y. Liang, W. Wang, H. Ding, M. Chen, J. Tang, B. Hooi, Structure-aware label smoothing for graph neural networks (2021). arXiv:2112.00499.

[41] Y. Xie, Z. Xu, J. Zhang, Z. Wang, S. Ji, Self-supervised learning of graph neural networks: A unified review, IEEE Transactions on Pattern Analysis and Machine Intelligence 45 (2) (2023) 2412–2429.

[42] J. Xie, X. Xiang, S. Xia, L. Jiang, G. Wang, X. Gao, MGNR: A multi-granularity neighbor relationship and its application in KNN classification and clustering methods, IEEE Transactions on Pattern Analysis and Machine Intelligence 46 (12) (2024) 7956–7972.

[43] S. Xia, X. Ma, Z. Liu, C. Liu, S. Zhao, G. Wang, Graph coarsening via supervised granular-ball for scalable graph neural network training, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025, pp. 12872–12880.

[44] S. Xia, G. Wang, G. Xu, S. Zhao, G. Wang, GBGC: Efficient and adaptive graph coarsening via granular-ball computing, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2025, pp. 3489–3497.

[45] W. Hoeffding, Probability inequalities for sums of bounded random variables, Journal of the American Statistical Association 58 (301) (1963) 13–30.

[46] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad, Collective classification in network data, AI Magazine 29 (3) (2008) 93–106.

[47] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, in: Relational Representation Learning Workshop, NeurIPS, 2018.

[48] D. Arthur, S. Vassilvitskii, k-means++: The advantages of careful seeding, in: Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, 2007, pp. 1027–1035.

[49] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[50] G. W. Brier, Verification of forecasts expressed in terms of probability, Monthly Weather Review 78 (1) (1950) 1–3.

[51] L. Hubert, P. Arabie, Comparing partitions, Journal of Classification 2 (1) (1985) 193–218.